

Project 1: Data Link Protocol

Implementation of a Data Link Protocol for Serial Cables

Application and Report made by

Deniz Lopes Gunes (up202208453)

Summary

The protocol was developed with a modular design and implemented a basic Stop-and-Wait Automatic Repeat Request mechanism, with the capability to discard invalid or repeated data packets while also repeating transmissions with dynamically set Timeout and Retransmission variables. In a clean channel, when the theoretical maximum efficiency (S) would be $S = 1.00$ the protocol's measured efficiency was around $S = 0.78$ which is a significant difference. I will analyse this finding in the report.

Introduction

The main objective of the lab was to design and implement a data link protocol that operates over a serial RS-232 link between two Linux-based computers. The application can be run in two modes, RX and TX.

Regardless of mode, the work is divided into two layers, independent of each other. The Application Layer and the Link Layer. They are independent from each other.

While in RX mode, the responsibilities of the Application Layer are to open the connection, after it's established keep reading data packets and writing them to a file until it gets the end control packet and then close the connection.

While in TX mode, the responsibilities of the Application Layer are to open the connection, after it's established read the file and send data packets until the end of the file, then send an end control packet and finally close the connection.

For debugging purposes both modes print out the transmission stats at the end.

Methodology

I followed a very modular system, having a separate file for each major task. I split all the major Link Layer functions into different files, each responsible for a Link Layer function.

Helpers

These functions are used across different files and may slightly differ in behaviour depending on the goal of the calling function.

compute_bcc2



```
frame_helpers.c

unsigned char compute_bcc2(const unsigned char *data, size_t length) {
    unsigned char bcc2 = 0;
    for (size_t i = 0; i < length; i++) {
        bcc2 ^= data[i];
    }
    return bcc2;
}
```

Figure 1: compute_bcc2 function

This function takes in an array of bytes and its respective length and computes its exclusive or.

send_frame

```
frame_helpers.c

int send_frame(const unsigned char *frame, size_t size) {
    if (writeBytesSerialPort(frame, size) < 0) {
        perror("writeBytesSerialPort");
        if (closeSerialPort() < 0) {
            perror("closeSerialPort");
        }
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

Figure 2: send_frame function

This function takes in an array of bytes and its respective length and writes those bytes to the serial port. This function is defined since the code to write bytes to the serial port is verbose and used across various places.

read_frame

```
frame_helpers.c

int read_frame(const unsigned char *expected, unsigned char *abortSignal) {
    unsigned char byte;
    State currState = START;
    (*abortSignal) = 0;
    while (currState != STOP && (*abortSignal) != 1) {
        int bytes = readByteSerialPort(&byte);
        if (bytes == -1) {
            perror("readByteSerialPort");
            if (closeSerialPort() < 0) {
                perror("closeSerialPort");
            }
            exit(EXIT_FAILURE);
        }

        if (bytes > 0 &&
            !processStateMachine(&currState, byte, abortSignal, expected, NULL)) {
            break;
        }
    }

    return currState == STOP;
}
```

Figure 3: read_frame function

This function takes in an array of bytes which represent the expected frame, and an abort signal. It then keeps polling the serial port until it reads bytes and then passes it to the processStateMachine. This function exists only when a sequence of bytes equal to the expected sequence is read, when there is an abort signal or when there is a problem in the state machine. It returns an int representing a boolean where True (1) means I read the expected frame and False (0) I didn't read it either because there was an error or because I aborted it (timed out).

processStateMachine

```
link_layer_state_machine.c

int processStateMachine(State *currState, unsigned char byte,
                        unsigned char *abortSignal,
                        const unsigned char *expectedHeader, int *bcc1_errors) {

    if (*currState == STOP || *abortSignal != 0) {
        return 0;
    }

    State newState =
        transitionState(*currState, byte, expectedHeader, bcc1_errors);

    if (newState != *currState) {
        *currState = newState;
    }

    return 1;
}
```

Figure 4: processStateMachine function

This function takes in a reference for the current state, the read byte, the abort signal, an array of bytes representing the expected header, and a nullable pointer to an int to be increased in case of bcc1 errors, the last being used solely for statistics and having no real implication on the protocol. An auxiliary function transitionState is defined where the actual logic of the state machine is processed.

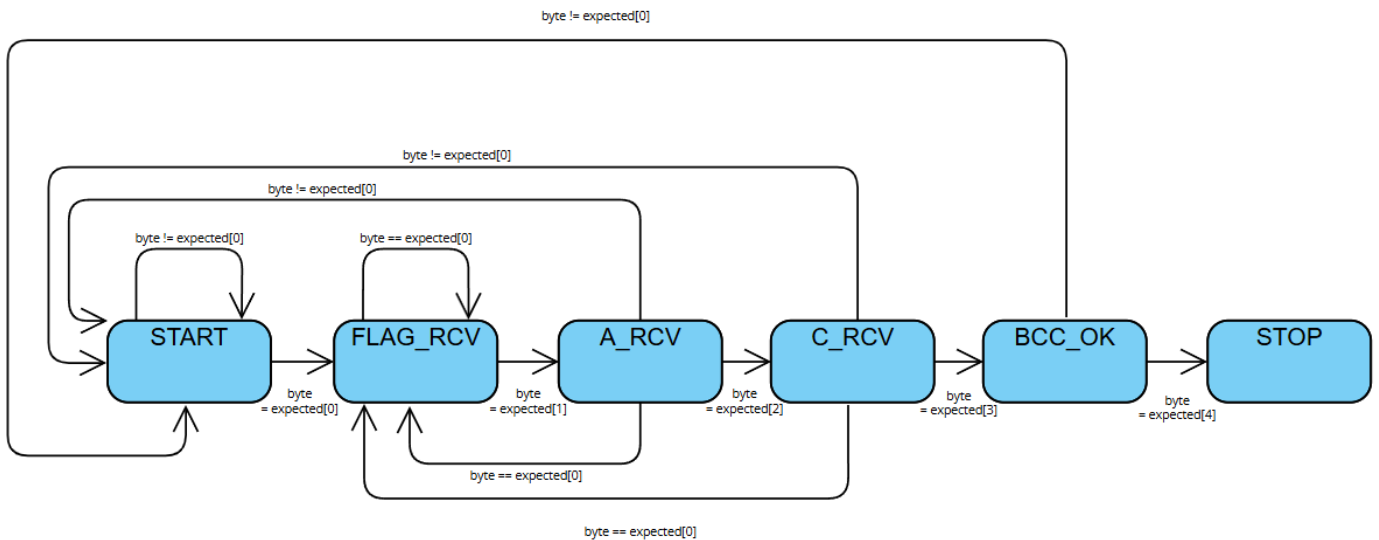


Figure 5: state machine

packet_helpers

```
packet_helpers.h

int build_control_packet(unsigned char* packet, const unsigned char* filename, int filename_length, int file_size);
int build_end_packet(unsigned char* packet, const unsigned char* filename, int filename_length, int file_size);
int build_data_packet(unsigned char* packet, const unsigned char* data, int data_length);
void parse_control_packet(const unsigned char* packet, int length, long* file_size, char* filename);
int parse_data_packet(const unsigned char* packet, int length, unsigned char* data, int* data_length);
```

These helper functions do what their names describe. Big-endian format was used to encode the length of the packet into two bytes L1 and L2. They are simple in nature and are made to build and parse packets to be sent / are received from the link layer. The parser don't handle byte destuffing as this is handled independently in the link layer.

Link Layer

ll_open

This function is responsible for establishing the initial connection between the RX and TX. It was implemented in the ll_open_connection.c file.

The function operates in two modes: receiver (Rx) and transmitter (Tx).

In Rx mode, the process begins by opening the serial port and listening for a SET frame from the transmitter. Once a SET frame is received, the receiver responds with a UA frame, indicating that the connection has been successfully established.

In Tx mode, the serial port is also opened, but the transmitter takes the initiative by sending a SET frame and waiting for a UA frame in response for a specified TIMEOUT period. If a UA frame is received within that time, the connection is considered established. If not, the transmitter resends the SET frame and repeats the process up to RETRANSMISSION times.

ll_close

This function is responsible for closing the connection. In a real implementation of this protocol my approach would have been different. Because the behaviour of our application is very predictable, it's very unlikely either the transmitter or receiver call ll_close wrongly. Because of this I can assume that once they are called, I want to close the connection regardless of if I get the DISC and UA from the transmitter. In either operation modes I send a DISC frame. After this in TX mode I wait for a TIMEOUT period until I read another DISC frame. If I don't get one, I still send a UA frame and close the serial port. The same happens in the RX. I send a DISC frame and wait to get a UA frame for a TIMEOUT period. If I don't, I still close the serial port. With this I avoid unnecessary retransmissions because in this application, if either one reaches the ll_close function, I want to close the connection.

ll_read

This is one of the main functions of the program as it handles the logic of reading the data packets. It handles byte destuffing, send REJ frames when I get a BCC2 error, send RR frames to acknowledge data packets and discards duplicate frames. The `processStateMachine` is used to wait for a header. The `expectedHeader` argument is the corresponding header to the current data packet number. If I'm ready for packet 0 I set the expected header to an I(1) frame. This is because the `readyFor` variable, which is used to check for duplicate frames, is set to the next frame's number. Meaning if `readyFor` is 1 I'm currently waiting for I(0). This logic is implemented inside a helper function called `read_header`. Read header also keeps track of a previous state. This is to prevent duplicate frames. If I ever read a frame corresponding to the wrong packet numbering, I detect it as a duplicate and discard the data.

After reading the header, I start reading the data, destuffing it. Destuffing works by watching for an escape character and then reading the next byte. The function then converts this to either a FLAG or an escape character. If the combination of the escape character and the next byte is not known, meaning it is not a stuffed byte, the function just appends this byte to the data. After reading a flag, I compute the `bcc2` and compare it to the byte before the flag. If they match, I send an RR frame with the numbering corresponding to the `readyFor` variable. If not, I send a REJ frame. This data is then passed back to the application layer via a reference, and the number returned is the size of the data packet, or an error code.

ll_write

This function is the other main function of the program. It handles sending data packets, byte stuffing and retransmissions. The logic is a bit simpler. For `RETRANSMISSION` times I run an auxiliary function `send_and_wait_frame`. This function takes the frame I want to send and the expected response among other parameters. I then send the frame and wait for a `TIMEOUT` period or until I get a response. If the response is the expected one, I return to the application and send the next frame. If it's the REJ frame I don't return out of the function and send again, while increasing the amount of tries. If I don't get any response after the `TIMEOUT` period, I resend the frame up to `RETRANSMISSION` times.

Application Layer

The application layer is responsible for managing the main execution flow of the program, coordinating file operations and communication with the link layer. It determines whether the program will act as a transmitter (Tx) or a receiver (Rx) based on the role specified at runtime.

At the beginning of execution, the program initializes the link layer with the provided configuration parameters, such as the serial port, role, baud rate, timeout duration, and number of retransmissions. It then calls `ll_open` to establish a connection between the transmitter and receiver. If the connection cannot be established, the program terminates.

In transmitter mode (Tx), the application layer is responsible for reading the file that will be sent. It first retrieves the file size and constructs a control packet containing metadata about the file, including its name and size. This control packet is sent through the link layer using `ll_write`, signaling the start of transmission.

Afterwards, the file is opened in binary mode, and its contents are read in chunks. Each chunk of data is encapsulated in a data packet, which is then sent to the receiver via `ll_write`. The process continues until the entire file has been transmitted. Once all data packets are sent, the transmitter builds and sends an end packet to signal the completion of the file transfer. Finally, it calls `ll_close` to terminate the connection.

In receiver mode (Rx), the application layer continuously listens for incoming packets using `ll_read`. When a control packet is received, it extracts the file information (name and size) using `parse_control_packet()` and creates a new file for writing. As data packets arrive, the receiver decodes them using `parse_data_packet()` and writes the extracted data to the file. When an end packet is received, it indicates that the transfer is complete. The file is then closed, and `ll_close` is called to end the session.

Statistics

An auxiliary file for keeping track of the transfer stats was created. This file keeps track of the total amount of packets, retransmissions, time of transfer among other values. At the end it prints these stats. To simulate a frame error ratio the following lines of code were added to the `ll_read` function.

```
llread_frame.c

if (rand() % 100 ≤ BCC1_ERROR_PROB - 1) {
    printf("[RX] BCC1 Simulated error\n");
    ll_stats.error_frames++;
    return -1;
}
if (rand() % 100 ≤ BCC2_ERROR_PROB - 1) {
    printf("[RX] BCC2 Simulated error\n");
    ll_stats.error_frames++;
    send_frame(readyFor = 0 ? REJ0_FRAME : REJ1_FRAME, 5);
    return -1;
}
```

Figure 6: frame error simulation

This works by generating a random number and by using a defined BCC1 error probability and BCC2 error probability. If I simulate a BCC1 error I return out of the ll_read and don't send anything, if I simulate a BCC2 error, on the other hand, I send a REJ frame. The frame error ratio is the probability of either BCC1 error occurring or BCC2 error occurring.

I can also simulate propagation delay T-PROP by setting the propagation time in milliseconds and then calling `usleep(2000*T-PROP)` to simulate the propagation of the incoming and outgoing frame.

I varied these values including the BAUDRATE to analyse how the efficiency evolves with these factors.

Findings

Using the stats function I varied the FER (frame error ratio) Propagation delay and baud rate.

These were the findings.

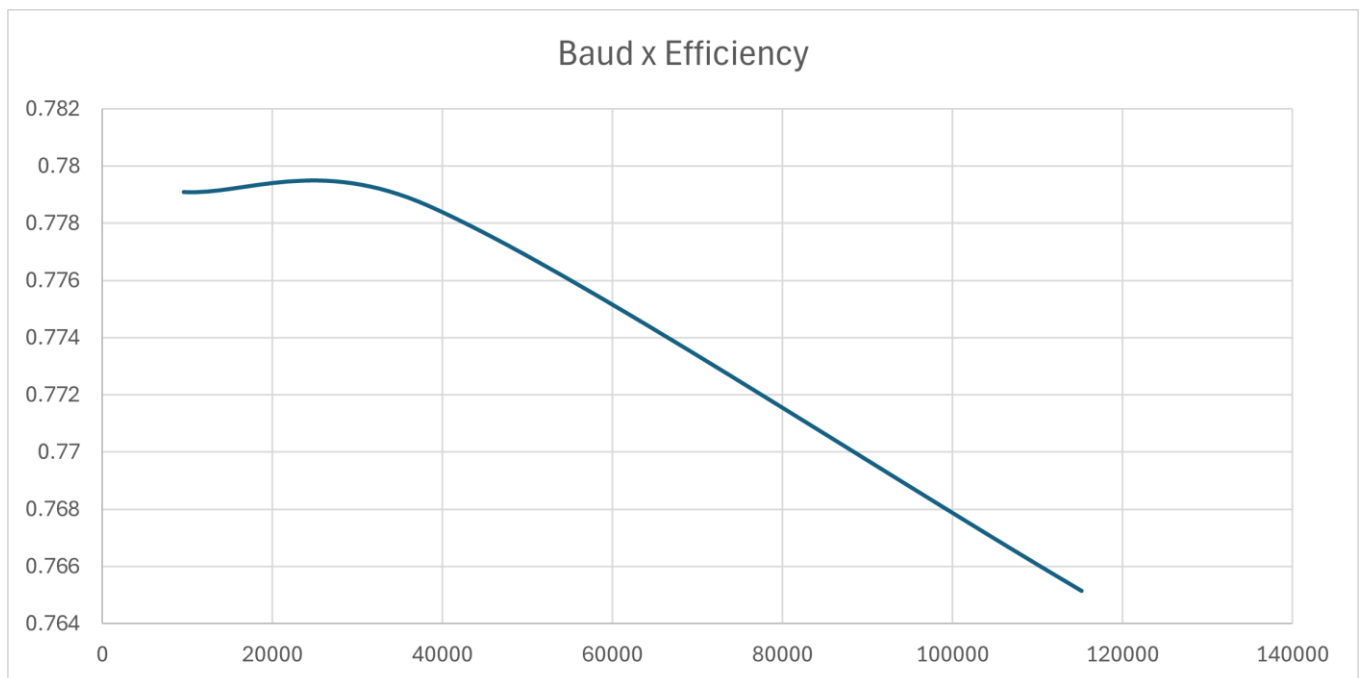


Figure 7: How efficiency varies with baud rate

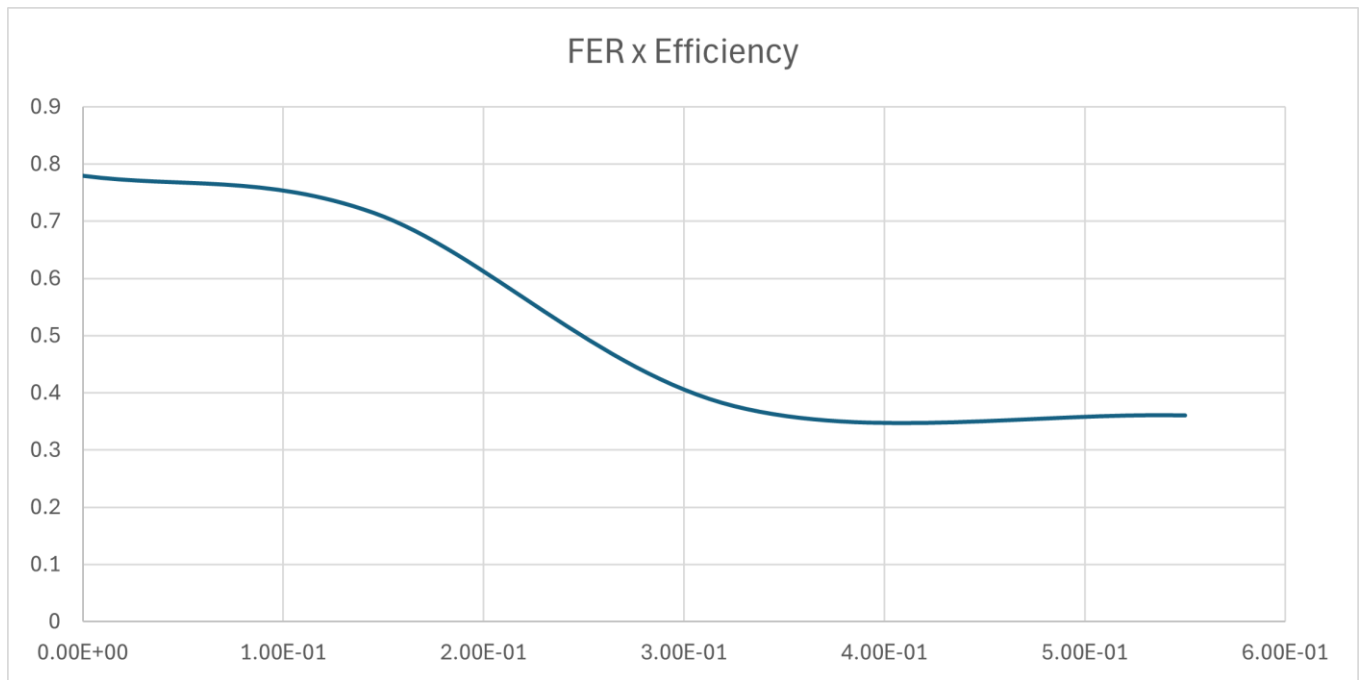


Figure 8: How efficiency varies with Frame Error Ratio

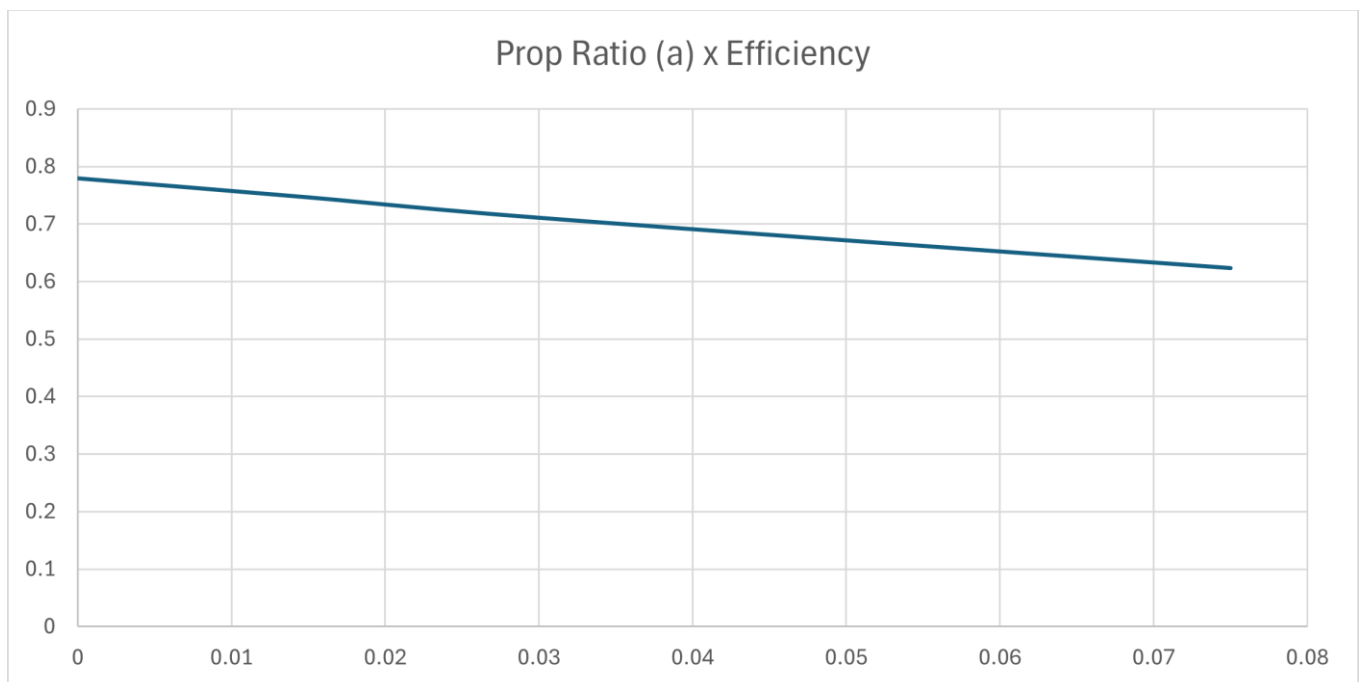


Figure 9: How efficiency varies with Propagation Time

Conclusions

The efficiency drastically falls with baud rate because as the transmission speed increases, the proportion of time spent on overhead, such as control frames, acknowledgments, and propagation delay, becomes more significant relative to the useful data being transmitted. In other words, while the baud rate determines how fast bits are sent, factors like timeouts, frame processing delays, and retransmissions do not scale proportionally, causing the effective data throughput to increase much less than the raw transmission speed. This results in a noticeable drop in measured efficiency at higher baud rates.

The efficiency decreases as the frame error ratio increases because a higher number of corrupted or lost frames leads to more retransmissions. Each retransmission consumes additional time and bandwidth without contributing to the transmission of new data. As a result, the effective throughput drops, since a larger fraction of the total transmission time is spent resending data rather than successfully delivering new information.

The efficiency also decreases with higher propagation delay because the system must wait longer for acknowledgments after sending each frame. This waiting time reduces the proportion of time spent actively transmitting data. In stop-and-wait protocols, this effect is especially noticeable, as the transmitter remains idle until it receives confirmation from the receiver. Consequently, longer propagation delays lead to reduced channel utilization and overall lower efficiency.